

1. Demonstrate use of git commands to push and pull 3 html files from Github account.

```
mkdir Demo
```

```
cd Demo
```

```
echo "<h1>Home Page</h1>" > index.html
```

```
echo "<h1>About Us</h1>" > about.html
```

```
echo "<h1>Contact</h1>" > contact.html
```

```
git init
```

```
git add .
```

```
git commit -m "Initial commit: Added 3 HTML files"
```

```
git remote add origin https://github.com/pro17/EXP1DEMO.git
```

```
git branch -M main
```

```
git push -u origin main
```

```
git clone https://github.com/pro17/EXP1DEMO.git
```

```
cd EXP1DEMO
```

```
git pull origin main
```

The screenshot shows the GitHub interface for a repository named 'EXP1DEMO' by user 'kaarthik'. The repository is public and has 1 branch (main) and 0 tags. The commit history shows a single commit titled 'Initial commit: Added 3 HTML files' by 'kaarthik' 4 hours ago. The commit details show three files: 'about.html', 'contact.html', and 'index.html', all added in the initial commit.

File	Commit Message	Commit Hash	Time
about.html	Initial commit: Added 3 HTML files	c8c6b4f	4 hours ago
contact.html	Initial commit: Added 3 HTML files	c8c6b4f	4 hours ago
index.html	Initial commit: Added 3 HTML files	c8c6b4f	4 hours ago

2. Demonstrate Building java program (addition of 2 numbers) through Jenkins.

📄 Jenkins Dashboard → **New Item**

📄 Name: AdditionPipeline

📄 Select **Pipeline** → OK

📄 In **Pipeline** section, choose "Pipeline script"

```
pipeline {
  agent any
  stages {
    stage('Compile') {
      steps {
        writeFile file: 'Addition.java', text: '''
        public class Addition {
          public static void main(String[] args) {
            int a = 10, b = 20;
            int sum = a + b;
            System.out.println("The sum is: " + sum);
          }
        }
        '''
        bat 'javac Addition.java'
      }
    }
    stage('Run') {
      steps {
        bat 'java Addition'
      }
    }
  }
}
```

📄 Paste the above Jenkinsfile content

📄 Click **Save** → Click **Build Now**

3. Demonstrate Continuous Integration process in Jenkins. Build a java program every 2 minutes (addition of two numbers) residing in Github repository.

Create Addition.java in GitHub Repository

```
public class Addition {  
    public static void main(String[] args) {  
        int a = 10, b = 20;  
        int sum = a + b;  
        System.out.println("The sum is: " + sum);  
    }  
}
```

Go to Jenkins dashboard → New Item → Enter a name (e.g., Java_Addition_CI) → Choose **Freestyle project** → OK.

🔗 Under **Source Code Management**, select **Git**.

Change master to main

🔗 Enter the **GitHub repository URL**: `https://github.com/your-username/your-repo.git`

🔗 Scroll to **Build Triggers**.

🔗 Check **Build periodically**.

🔗 Enter this in the schedule box: `H/2 * * * *`

Go to **Build** → Add build step → **Execute shell** (Linux/macOS) or **Execute Windows batch command** (on Windows).

```
javac Addition.java
```

```
java Addition
```

The screenshot shows the Jenkins configuration page for a new Freestyle project. The 'Source Code Management' section has 'Git' selected. The 'Repository URL' is set to 'https://github.com/kaarthikpro/Addition.java.git'. The 'Branch Specifier' is set to '*/main'. In the 'Build Triggers' section, 'Build periodically' is checked, and the 'Schedule' is set to 'H/2 * * * *'. In the 'Build Steps' section, 'Execute Windows batch command' is selected, and the 'Command' field contains the following text:

```
javac Addition.java  
java Addition
```

4. Demonstrate CI/CD in Jenkins. Consider 3 html files in Github repository and write a scripted pipeline to deploy on Xampp server.

<https://github.com/username/html-website>

```
|  
|-- index.html  
|-- about.html  
|-- contact.html
```

```
<!DOCTYPE html>  
<html>  
<head><title>About</title></head>  
<body><h1>About Us</h1></body>  
</html>  
  
<!DOCTYPE html>  
<html>  
<head><title>Contact</title></head>  
<body><h1>Contact Us</h1></body>  
</html>  
  
<!DOCTYPE html>  
<html>  
<head><title>Home</title></head>  
<body><h1>Welcome to the Home Page</h1></body>  
</html>
```

Pipeline script:

```
node {  
    stage('Clone Repository') {  
        git credentialsId: 'github-pat', url: 'https://github.com/kaarthikpro/EXP-4-LAB-EXAM.git', branch: 'main'  
    }  
  
    stage('Deploy to XAMPP') {  
        def xamppPath = 'C:/xampp/htdocs/mywebsite'  
  
        // Create directory if not exists  
        bat "if not exist \"${xamppPath}\" mkdir \"${xamppPath}\""  
  
        // Copy HTML files  
        bat "copy index.html \"${xamppPath}\"\\\\"  
        bat "copy about.html \"${xamppPath}\"\\\\"  
        bat "copy contact.html \"${xamppPath}\"\\\\"  
    }  
  
    stage('Deployment Completed') {  
        echo 'Website deployed successfully to XAMPP!'  
        echo 'Visit http://localhost/mywebsite/index.html to view it.'  
    }  
}
```

6. Write a Selenium script to perform automated testing.

```
# test_script.py

from selenium import webdriver
from selenium.webdriver.common.keys import Keys
import time
driver=webdriver.Chrome()
driver.get("https://www.google.com")
search_box=driver.find_element("name","q")
search_box.send_keys ("Ramrao Adik institute of technology")
search_box.send_keys (Keys.RETURN)
time.sleep(10)
```

5. Demonstrate CI/CD in Jenkins. Consider 3 html files in Github repository and write a declarative pipeline to deploy on Xampp server.

1. Go to **Jenkins Dashboard** → **Manage Jenkins** → **Credentials** → **(Global)**.
2. Click **"Add Credentials"**.
3. Select:
 - **Kind:** Username with password
 - **Username:** GitHub username
 - **Password:** GitHub Personal Access Token (PAT)
 - **ID:** github-creds
4. Save.



3. Create New Pipeline Job

1. Go to Jenkins Dashboard.
2. Click **"New Item"**.
3. Enter a name: DeployHTMLtoXAMPP.
4. Choose **Pipeline**.
5. Click **OK**.

Pipeline script:

```
pipeline {
  agent any

  environment {
    DEPLOY_DIR = 'C:\\xampp\\htdocs\\mywebsite'
  }

  stages {
    stage('Clone Repository') {
      steps {
        git credentialsId: 'github-creds', branch: 'main', url: 'https://github.com/kaarthikpro/EXP-4-LAB-EXAM.git'
      }
    }

    stage('Prepare Deployment Directory') {
      steps {
        bat """
          if not exist "%DEPLOY_DIR%" mkdir "%DEPLOY_DIR%"
        """
      }
    }

    stage('Deploy HTML Files to XAMPP') {
      steps {
        bat """
          copy /Y index.html "%DEPLOY_DIR%"
          copy /Y about.html "%DEPLOY_DIR%"
          copy /Y contact.html "%DEPLOY_DIR%"
        """
      }
    }

    stage('Deployment Completed') {
      steps {
        echo "Visit: http://localhost/mywebsite/index.html"
      }
    }
  }

  post {
    success {
      echo '✅ CI/CD completed successfully!'
    }
    failure {
      echo '❌ Deployment failed.'
    }
  }
}
```

7.Create an image of php project and push on Dockerhub repository.

Folder:

Dockerfile :

```
# Use PHP with Apache
FROM php:8.2-apache

# Set working directory
WORKDIR /var/www/html

# Copy PHP file to Apache root
COPY hello.php .

# Set correct permissions
RUN chown -R www-data:www-data /var/www/html \
    && chmod -R 755 /var/www/html

EXPOSE 80
```

hello.php :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>PHP Docker</title>
</head>
<body>
  <h1><?php echo "Hello world from a PHP container"; ?></h1>
</body>
</html>
```

```
docker build -t pro17/php-project .
docker login
docker push pro17/php-project
http://localhost:5000/hello.php
docker build -t php-docker-demo .
docker run -d -p 5000:80 php-docker-demo
```

<http://localhost:5000/hello.php>

